

Assessment 2

Group 10 Members:

**Ben Walker, Josh White, Chris Williams, Gavindu Tissera, Nana
Twum Oviarobo, Tamerlan Urazbayev, Tymur Topala**

Change Report

Project Tracking Methodology

For the second part of our project it was imperative to keep track of where changes were being made to previous work, and also our additions to the project. Various tools were utilised throughout the lifecycle of the second stage of development to achieve this; these are discussed below:

Weekly Meetings: Our weekly meetings provided the opportunity for the planning and discussion of proposed changes to the inherited documentation. Whilst still at University before the Christmas break, it was essential that the team members responsible for compiling the change report elicited feedback from other members of the group so that a broad range of perspectives were considered for the report. This raised some differences of opinion which prompted thoughtful and constructive discussion that ensured that any proposed changes were consistent with the view and agreement of the team.

Effectively utilising the change report: The creation and proper use of our change report constitutes the majority of our methodology, as it provides a platform for justifying and reasoning any alterations made to the inherited documents. It allows other team members to question and discuss the improvements made by others, which lends to a more collaborative and representative set of documentation as all members are able to review and contribute to the report. Were changes not to be documented, it could lead to discrepancies in our final deliverables, as teammates would be unable to review each other's contributions. For these reasons, it is essential that this document comprehensively discusses all changes made to the original documentation and that these changes are justified suitably.

Google Drive: Following on from stage one of the assessment, we identified that the use of Google Drive was of huge benefit to us and that its use should extend into the second stage. This application allows us to retain all documentation in a singular, shared drive and ensures that any changes made to these documents are consistent across every member's device. Moreover, Google Docs provides a review utility that allows members to suggest changes to documentation without a direct alteration, further encouraging collaboration during this process. By utilising this effectively, the changes made during the latter stages of development should represent the consensus of the team.

Github Versions: Github is another example of a resource that proved invaluable during the first stage of development, granting access to previous versions of our repository and maintaining a log of all changes to our source code. We configured this version control software so pull requests are required to receive an accepting review from another team member, ensuring that the changes made are safe and validated. The previous developers of the project also adopted this software, emphasising its value as a tool for tracking and reviewing change.

Documents - Red Text: To keep track of changes to our inherited documents, we chose to use red font for new textual additions and in the captions of any new UML diagrams. This visual difference unequivocally distinguishes our inclusions from the existing documentation and was decided as the most appropriate of formatting our updates to these documents.

Changes to Method Selection and Planning

Within the [original method selection and planning deliverable](#), it was challenging to identify any parts that could be changed or modified without removing important information from the earlier stages of the development of the project. The initial plan is descriptive and we were impressed with the detailed discussion of role allocation and the methods behind this process. Moreover, the quality of the codebase inherited indicates the initial ideas and discussions within this document are of sound reasoning and require no alteration. The only modification made directly to the original document was to include an accurate link to the Gantt charts described in phase 6, as this previously held a link to the repositories website with no indication of where the diagrams could be accessed. We also decided to make a single gantt chart displaying our development process, as opposed to the weekly updates previously implemented, as weekly updates are already present in our plan. These changes can be seen in the [updated method selection and planning document](#).

In addition to this physical change, a methodological change was implemented as well by transitioning from the Scrum methodology, employed in the first stage of development, to DevOps. This change was made upon considering the reasoning within the original document compared to our own experiences in development during the first stage of the project. The considerations surrounding these changes are discussed below:

- **Emphasising Automation:** The DevOps methodology focuses on utilising automation to its full potential, thus reducing the time required for software to be delivered. During our first stage of development, we employed the use of GitHub actions to automate the building, testing and deployment of our software. By automating these processes we were able to free resources and divert attention to implementing functionality within the game, instead of performing time consuming tasks. Because of this, we decided it would be beneficial to adopt a methodology that prioritised this automation for the second stage of development as it would allow us to focus solely on implementing the requirements for our project whilst reducing manual errors in our code.
- **Performance Benefits:** Research has proposed that DevOps cultural approaches to software development yield fewer defects, higher frequency deployments of software and fewer system risks during development [1, pg. 4]. It is important to note that the research cited considers a comparison of 3 different Java projects, so it could be argued that the study lacks statistical power, however, due to the scale of our project it would be rational to suggest that the CI/CD at the core of DevOps would provide a measurable benefit to the rate at which we can release stable software.
- **Considerations from previous assessment:** At the beginning of assessment one, we identified that adopting the Scrum methodology could potentially lead to issues in relation to teamwork aspects of the project. We believed that the assignment of a Scrum master was not necessary, given that each individual's strengths and weaknesses in relation to the role were unknown at this time. Moreover, by equally distributing the responsibility during weekly sprints, workloads became far easier to manage as each team member was familiar and confident with their assigned roles and no individual was overburdened with organising and dictating our meetings. Our approach to assessment 2 can be summarised as a hybrid of DevOps and Agile as we

continued to employ weekly meetings to promote and encourage teamwork and collaboration.

Changes to Requirements

After reviewing the [inherited requirements document](#), it is clear that the previous elicitation methodology is sufficient in both detail and reasoning, referencing studies to support the choice of register format and discussing further research to supplement the client interview. An improvement we felt was necessary, however, was to justify the use of the 'should', 'shall' and 'may' as descriptors for the priority of each requirement, as this is not included in the original document. In the [updated requirements document](#) a reference has been included to support this choice of formatting.

Issues within the original document primarily occur within the requirement register, as there are some discrepancies with the categorisation of functional and non-functional requirements, as well as misleading or inaccurate requirement ID's. It is understood that a functional requirement defines what functionality and operations the system has, whereas non-functional requirements denote how the system should perform. With this in mind, our alterations and their justifications are discussed below:

- ***NFR_FAST_LOAD → FR_FAST_LOAD : Moved to functional requirements*** - Asset loading and rendering is a functional requirement that the system must implement through methods and assets. As a collective, we were confused as to why this requirement was initially listed as non-functional as this functionality can be directly measured through testing. For this reason, the ability for the game to load and render assets before the user has made an input has been reclassified as a functional requirement.
- ***NFR_PLAYABLE → FR_FUNCTIONAL : Moved to functional requirements*** - The playable requirement is described as "the combination of all of the functionality must lead to a game which can be played from start to finish". We believe this fits the functional requirements more appropriately as it describes a collation of functional components that allow the game to fulfil its purpose. We also thought the title 'playable' was inappropriate given the description attributed to it. The previous title could be referencing the user experience of playing the game, an instance of a non-functional requirement, however the description does not conform to this idea. For this reason we moved it to the functional requirement table whilst also renaming it to FR_FUNCTIONAL, which we felt was a far better reflection of the requirement and its purpose.
- ***NFR_FORGIVING → FR_BUILDING_EDIT : Moved to functional requirements*** - The NFR_FORGIVING requirement reads as follows, "the game will allow the player to alter previous decisions and mistakes, such as misplaced buildings". The initial reason for changing this was due to the property 'forgiving' being vague in its relation to the system as it is difficult to identify and measure this property due to the differing opinions and expectations that users might have regarding this. Moreover, the description doesn't lend itself to being a non-functional requirement, instead a

functional requirement that we can directly implement within the system. For this reason, the requirement was moved to the functional requirements section and renamed for consistency and clarity in its intent and purpose.

- ***NFR_CONCURRENCY → FR_CONCURRENCY : Moved to functional requirements***
 - The description assigned to the original requirement reads: “the game should be able to handle both mouse and keypresses that may be occurring at the same time and possibly conflicting”. As a group we concluded that this was a functional requirement due to the fact it isn't a trait of the game and instead something that has to be directly implemented into our code. For this reason it has been moved to the functional requirement table.
- ***FR_PLACE_BUILDING - “The user will be able to place down buildings on the map” → “The user will be able to place multiple buildings of varying type on the map”*** - As the scope of the project increased in assessment 2, we felt it appropriate to update this requirement to reflect the new expectations for the project. The initial requirement was sufficient for the delivery of assessment 1, as only a select number of buildings were mandatory, however as priorities shifted we recognised the importance of reflecting this change in the requirements table so the documentation was consistent with the functionality implemented in our software.
- ***UR_SETTINGS - “Should” → “May”***: The initial settings requirement suggested that this component was of high importance and would feature in the final version of the software, however we felt this fell outside the scope of the project and the expectations for the second stage of the project's life cycle. Additionally, this feature was unimplemented on inheritance of the project with no reference to its omission in the [implementation report](#) provided on the website. Due to time constraints and the necessity for more complex functionality to be implemented, we have changed the priority of the original requirement as a reflection of this.
- ***UR_TEACH_DURING_GAME → UR_INTUITIVE_UX***: The ID of this requirement was changed due to the misleading nature of its original format. The initial ID suggests that the game should contain a learning element as part of the user experience, which falls entirely outside the scope of the product brief and description given to the requirement. We believed that changing the name to ‘UR_INTUITIVE_UX’ to more accurately reflect the initial intentions and idea behind the requirement. Whilst not a drastic change in isolation, were this requirement ID to be referenced throughout further documentation during the project's subsequent development, it could have portrayed an inaccurate representation of the game's functionality.
- ***FR_TEXTUAL_DESCRIPTIONS - “The game provides textual descriptions during gameplay to help the user understand how to play” → “The game provides basic information about the building's price, type and size when previewed in the buy menu”***: This requirement was based on ‘UR_TEACH_DURING_GAME’ and changed due to the vague wording of the original description, and to better fit the user requirement changing to ‘UR_INTUITIVE_UX’. We believed that the phrase “textual descriptions” wasn't specific enough, so we included the individual factors (price,

building type, and size) that we believed more accurately reflected the original intentions of the requirement.

- ***NFR_SIMPLE* → *Fit criterion changed***: The original fit criterion for this requirement denoted that ‘users should be comfortable playing the game within 60 seconds’. We felt this criterion lacked clarity and provided a vague reference of measurement. Just as it is difficult to measure an abstract concept such as ‘simplicity’, it is equally difficult to measure ‘comfort’. Furthermore, this criterion ignores the accessibility needs of users through its ‘60 second benchmark’. What one person can achieve in this time frame may exceed or fall short of another person’s ability, as simplicity is a subjective opinion. We felt it would be appropriate to redefine the fit criterion for this risk with measurable constructs that we can implement in the game to fulfil this requirement. The updated criterion reads: ‘The screen should contain no more than 6 separate UI elements to reduce the complexity of the game’. This provides a measurable criteria for the requirement and appropriately addresses the concerns discussed above.
- ***FR_HOVER_INFO* → *Removed***: We decided to omit this requirement from the documentation upon the refactoring of the *NFR_SIMPLE* fit criterion. We felt it was counterintuitive to provide such a detailed feature in the game’s current stage of development given the desire for the game to be simple and intuitive in nature. We felt by providing individual UI components for each of the buildings, it would potentially overburden users and might not reveal any new information that could not be inferred from the satisfaction scale already provided. For this reason, the requirement was removed during this stage of development.

Aside from these changes, we also included new user and functional requirements to reflect the additional requirements included within the Assessment 2 brief. This involved “UR_ACHIEVEMENTS”, “UR_LEADERBOARD” and the functional requirements based off of them, all listed in red.

Changes to Risk Assessment

The inherited risk assessment provides a descriptive overview of the identification and management of risks in the foreword, however the risks provided lack clarity and appropriate depth in relation to the specifications outlined. The overall register format is accessible and well formatted and therefore does not require modifications, however some of the individual risks and justifications have been altered to provide a more appropriate level of description, these are discussed below:

- ***Removal of ‘Leader’ ownership***: As our methodology has transitioned from Scrum to DevOps, we felt it was unnecessary to keep the leader ownerships specified in the [original risk assessment](#). Instead we have assigned these ownerships to the entire team as a core part of our approach to this assessment is shared accountability and responsibility.

- **R2 : ‘Timeline slippage, time is wasted’ → ‘Concerns and progress are not adequately addressed in weekly sprints, leading to setbacks in meeting deadlines’** : This risk was changed as it lacked clarity and meaning in its initial form. We felt it appropriate to link this risk to the software development methodology imposed at the time of writing, which was Agile, thus making the risk relevant for the project and the practices adopted in the first stage of development. By contextualising the initial risk in a specific environment - weekly sprints - it becomes easier to monitor and act upon, which is outlined as a key factor of the risk management process in the initial risk assessment.
- **R4: ‘Getting bugs in the code’ → Mitigation changed** : The original mitigation for this risk stated ‘check the code and figure out where the bug is’, which provides limited information on how this risk should be monitored and addressed. The project utilises the GitHub version control software which provides users with the ability to review each other’s pull requests, ensuring any code that may express unexpected behaviour at runtime is caught and changed before being merged to the main branch. Moreover, appropriate unit tests would ensure that the code functions and behaves in a secure and expected manner. Both of these considerations provide a tangible way with which to ‘figure out where the bug is’ and link suitably to the software used to develop the project. As such, the mitigation has been altered with this in mind in the updated deliverable.
- **R5: ‘Having engine limitations, for example: A computer cannot run a program because of its system.’ → ‘Program exceeds basic hardware requirements outlined by the stakeholder’** : The decision to refactor this risk was made due to confusion surrounding its initial wording. Engine limitations / operating system limitations are not a likely issue given the JVM, allowing Java bytecode to be run on any operating system. Moreover, it is outlined in [the initial requirements document](#) that multiple operating system compatibility is a functional requirement for the game and is also ranked with the highest degree of importance for the system, therefore it is highly unlikely that the games implementation would compromise this. We felt a more appropriate issue to address is the hardware requirements of the system, as the game is required to run on a minimal specification system. Whilst still an unlikely possibility given the scale of the project, it is still a concern given potential memory leaks and improper disposal of assets. In addition, the mitigation for the original risk was to ‘use another computer’, which is a direct contradiction to the expectations for the game outlined in [FR_OPERATING_SYSTEMS](#) as using another device does not resolve the problem or provide compatibility across systems. As such, the mitigation for the refactored risk has also been changed.

We have extended the document with newly identified risks throughout the second half of the project. These risks follow from R10 in the [updated risk assessment](#).

Changes to Architecture

Upon review of the [initial architecture document](#) on Team 3’s website, we noticed that there was no inclusion of an extensive class diagram depicting the entities in the system and how they interact, nor were there any interim diagrams to be observed on the website that portray the evolution of the project and how the final release was obtained. Due to the lack of

descriptive diagrammatic representation, the only way to understand the interactions occurring within the system is to manually parse the source code, which is inefficient.

Moreover, the only class diagram presented in the document provides only a very basic overview of the ECS used in the game, and it is not immediately apparent what relationships and compositions are implemented between entities within the game. Furthermore, the class diagram has some discrepancies with UML standardisation, resulting in an ambiguous class diagram.

The initial documentation outlines that *'chequered lines represent the parent class of building to its children subclass'*, however UML standards dictate that inheritance is modelled with a dotted line and an open arrow connecting the child class to its parent [2]. Furthermore, there is no discussion of what these arrows represent and what relationships classes hold between each other. The current relationship modelled between the building types is said to be inheritance however this is actually a dependency relation that is modelled. The methods outlined in the class diagram are primarily getter and setter methods, which can be assumed to exist through the attributes outlined and are not a necessary inclusion in the diagram. In addition the signature of these methods does not specify a return type, which is required by convention. This can also be seen in the attributes of the 'GameScreen' class which also have no specified attribute types and Tile which specifies an attribute type as 'png', whereas it is actually an instance of the libGDX 'Texture' class.

Another issue with the initial class diagram is the representation of the buildings, which cannot be said to follow any conventional UML representation, as no attributes or methods are listed. By reviewing the source code it can be seen that the 'Building' class is actually a composition of the 'BuildingType' class which uses the 'BuildingCategory' Java 'enum' class. With this information considered, we have produced a more appropriate remodelling of the class diagram in figure 3 which is included in the updated architecture report.

The changes made to the original diagram are discussed below:

- **Addition of Screen Interface:** The GameScreen class is a realization of the libGDX 'Screen' interface, which contains the 'show' method. This method was initially shown to be a method of the GameScreen class itself, however, it is only overridden in this class and originates from the Screen interface provided by libGDX.
- **Composition relationships:** The relationships modelled in the initial class diagram denoted directed associations from the MainTimer to the GameScreen, Tile to GameScreen and Building to Tile. These associations were originally modelled with incorrect directionality, i.e. the MainTimer does not use the GameScreen, but rather the opposite. Moreover, some of the relationships modelled were not as strong as they should have been. For example, the GameScreen is *'composed'* of a timer and the timer only has meaning within the container of the GameScreen class - the GameScreen class is responsible for the creation of the MainTimer and if the GameScreen were to be deleted, the timer would cease to exist also. A similar alteration has been made between the Building and Tile classes and the BuildingType and TileType Enum classes. The former classes are responsible for the instantiation and deletion of the latter classes, and through the consistent naming of constructs

within the source code it is sufficient to say that a Building 'has-a' BuildingType to justify this composition relation.

- ***Inclusion of Enum classes:*** the source code for the system makes heavy use of the Java Enum class constructs. PlantUML [2] provides a representation for this class type which allows these structures to be explicitly shown in the improved diagram. In addition, this shows how the Building class is composed using these structures and represents a more accurate ECS arrangement for the system, as was outlined in the architecture document.
- ***Consistent method signatures:*** all methods included in the new diagrams have their full signatures and return types specified, as per UML convention.
- ***Omission of getter and setter methods:*** in the new diagram, the original getter and setter methods have been replaced with more functional methods within the classes. As well as this, a complete set of attributes is specified for the included classes, providing a more detailed overview of the function of each class. The existence of these getter and setter methods can still be inferred through the presence of correct directional associations and composition relationships.
- ***Inclusion of the World class:*** The initial diagram would suggest that the Tile class is responsible for placing the buildings on the map, however by viewing the source code, the class responsible for this functionality is actually the World class, which is not modelled. By including this in the architecture, further clarity is provided over how the classes are used in order to implement this functionality in the game.

In addition to the updated class diagrams, we have also provided a state and sequence diagram for stage one of the project's development, which can be found in our UML PDF on the website, and in the [updated architecture document](#). These diagrams were absent from the original document, therefore we felt it necessary to retrospectively create them to provide further clarity around the system's functionality and allow us to extend these diagrams further alongside our own implementation in stage two of the project.

References

- [1] P. Narang and P. Mittal, "Performance Assessment of Traditional Software Development Methodologies and DevOps Automation Culture", *Eng. Technol. Appl. Sci. Res.*, vol. 12, no. 6, pp. 9726–9731, Dec. 2022.
- [2] 'Class Diagram', plantUML.com,
<https://plantuml.com/class-diagram#:~:text=QA%2D16258%5D-.Relations%20between%20classes.-Relations%20between%20 classes> (last accessed 25th November 2024)